Technical Analysis

1. Purpose

FinWallet's technical goal is to provide a financial-backend example in which money movements are correct, safely retryable, traceable and resilient to external-service failures. The system is designed not only for happy paths but also for duplicate requests, concurrency, provider timeouts, fraud failures, session revocation and data-consistency problems.

2. Functional scope

Currently implemented:

- customer registration + OTP verification;
- login, JWT access token, refresh-token rotation and server-side sessions;
- TRY/USD/EUR wallet create/list;
- external bank-account opening through FakeBank;
- internal/external fraud evaluation;
- durable idempotent wallet-to-wallet transfer;
- double-entry ledger;
- client and provider traffic through YARP Gateway;
- Swagger, rate limiting and shared HTTP security controls.

Planned but not yet complete:

- BankDeposit;
- BankWithdrawal;
- merchant purchase/campaign accounting;
- public refund/reversal flows;
- durable manual fraud review;
- outbox/inbox;
- reconciliation;
- transaction-history read model.

3. Critical quality requirements

Financial correctness: Wallet balance, FinancialTransaction, IdempotencyRecord and Ledger changes must commit consistently inside the same financial transaction boundary.

Idempotency: Retrying the same money-changing operation must not move money twice. Reusing the same key with a different payload must produce a conflict.

Concurrency: Concurrent spending from one wallet must not cause overspending. MSSQL is the final authority.

Security: Public traffic passes through Gateway JWT validation, while service-level JWT/ownership is validated again. Provider routes use separate internal and downstream service credentials.

Availability: Provider timeout or 5xx responses must not corrupt financial correctness. Mandatory decisions such as fraud evaluation fail closed.

Auditability: Financial movements must be explainable through ledger entries and immutable business-transaction records.

4. Data authorities

- **MSSQL:** durable source of truth for customer, authentication/session, wallet, bank account, transaction, idempotency and ledger state.
- **Redis:** transient state such as OTP challenges; never the authority for money correctness.
- **Fake providers:** simulate external systems and cannot directly mutate FinWallet tables.

5. Runtime boundaries

```
Client -> YARP Gateway -> FinWallet.Api
FinWallet.Api -> YARP Gateway -> Fake providers
```

The Gateway owns routing, edge security and traffic controls. Business authorization and financial rules remain in the application/domain layers.

6. Performance approach

- SqlClient connection pooling is used.
- Redis ConnectionMultiplexer is singleton.
- HttpClientFactory + SocketsHttpHandler pooling is used.
- YARP destination connections, timeouts and load balancing are configuration-driven.
- SQL isolation/locking required for financial correctness is not weakened merely for benchmark throughput.

7. Testing approach

Moq is used only for Application orchestration boundaries. Real MSSQL locking, Redis Lua and YARP routing are not considered proven by mocks; they require integration/concurrency tests.

8. Most important current gap

A new wallet starts with zero balance and no public BankDeposit endpoint currently exists. Therefore a fully public-API-only register -> fund -> transfer path is not yet possible. This is the most important next functional gap in the project.